What is Database Testing and why it's important

What is Database Testing?

Database Testing is a type of software testing where we verify the **database layer** of an application. It ensures that data is **stored, retrieved, updated, and deleted correctly** in the database.

It mainly checks:

- ✅ Data integrity
- ✅ Data consistency
- ✅ Data accuracy
- ✅ Database schema (tables, columns, data types)
- ✅ Stored procedures, triggers, functions
- (**Trigger → Runs automatically when table data changes.
  Function → Takes input and returns a calculated value. **)
- ✅ Constraints (Primary Key, Foreign Key, NOT NULL, etc.)

In simple words:
Database Testing makes sure the backend data is correct and reliable.

Why is Database Testing Important?

1️⃣ Ensures Data Accuracy

If wrong data is stored, the whole system can fail. For example, wrong salary calculation or wrong student GPA.

2️⃣ Maintains Data Integrity

It verifies:

- Primary key is unique
- Foreign key relationships are correct
- No duplicate or invalid data

3️⃣ Prevents Data Loss

Testing ensures:

- Data is not lost during transactions
- Rollback works properly

4️⃣ Validates Business Logic

Many business rules are written in:

- Stored Procedures
- Triggers

Database testing checks if those rules work correctly.

5️⃣ Improves Application Performance

It checks:

- Query performance
- Index usage
- Execution time

Slow database queries = slow application.

6️⃣ Enhances Security

Ensures:

- Unauthorized users cannot access sensitive data
- Proper permissions are set

Example (Simple Scenario)

Suppose a student registration system:

- When a student named **Binti** registers
- Data should be saved correctly in the `Students` table

- Student ID must be unique
- GPA must be stored in correct format (e.g., 3.36 not text)

Database testing will verify all of this.

Short Definition

Database Testing is the process of validating database schema, tables, stored procedures, triggers, and data integrity to ensure correct data storage and retrieval.

◇ Types of Databases: Relational vs NoSQL

There are mainly two popular types of databases:

1. **Relational Database (RDBMS)**
2. **NoSQL Database**

1️⃣ Relational Database (RDBMS)

A **Relational Database** stores data in **tables** (rows and columns).

- Data is structured
- Tables are connected using **Primary Key** and **Foreign Key**
- Uses **SQL (Structured Query Language)**

☑ Examples:

- MySQL
- Oracle Database
- Microsoft SQL Server
- PostgreSQL

Features:

- Fixed schema (structure must be defined before inserting data)
- Strong data consistency

- Supports ACID properties
- Best for structured data

🔎 Example:

Students Table:

StudentID (PK)	Name	GPA
101	Binti	3.36

Courses Table:

CourseID (PK)	StudentID (FK)	CourseName

Here tables are connected using Foreign Key.

🎯 Best Used For:

- Banking systems
- University systems
- ERP systems
- Any system needing strong consistency

2️⃣ NoSQL Database

NoSQL means "Not Only SQL".

It does NOT store data in traditional tables.

It can store:

- Documents
- Key-Value pairs

- Graphs
- Wide-columns

☑ Examples:

- MongoDB
- Cassandra
- Redis
- Firebase

📌 Features:

- Flexible schema
- Handles large amounts of data
- High scalability
- Good for unstructured or semi-structured data

🔑 Example (MongoDB Document):

```
{
  "studentID": 101,
  "name": "Binti",
  "gpa": 3.36,
  "courses": ["CSE101", "CSE102"]
}
```

No need for separate tables.

Main Differences

Relational	NoSQL

Table-based	Document / Key-Value / Graph
Fixed schema	Flexible schema
Uses SQL	May not use SQL
Strong consistency	High scalability
Best for structured data	Best for big/unstructured data

🎓 Short Answer

- **Relational databases store structured data in tables using SQL and maintain strong relationships between data.**
- **NoSQL databases store unstructured or semi-structured data with flexible schema and high scalability.**

1️⃣ Overview of SQL Databases

SQL Database (Relational Database) stores data in **tables** (rows & columns).

- Structured format
- Uses **SQL (Structured Query Language)**
- Supports relationships between tables
- Follows **ACID properties**

☑️ Popular SQL Databases:

- MySQL
- PostgreSQL
- Microsoft SQL Server
- Oracle Database

👉 Best for banking systems, university systems, ERP, etc.

2️⃣ Introduction to MySQL

MySQL is:

- Open-source relational database
- Very popular and beginner-friendly
- Uses SQL language
- Widely used in web applications

Example: Many websites use MySQL with PHP.

3️⃣ Understanding Tables, Rows, Columns

Imagine a **Students** table:

StudentID	Name	GPA
101	Binti	3.36

◇ Table

Collection of related data.

◇ Row

One record.
Example: (101, Binti, 3.36)

◇ Column

One attribute.
Example: Name, GPA

👉 Table = Data container
👉 Row = Record
👉 Column = Field

4️⃣ Basic SQL Queries

◇ SELECT (Read Data)

```
SELECT * FROM Students;
```

👉 Shows all data.

```
SELECT Name FROM Students;
```

👉 Shows only Name column.

◇ INSERT (Add Data)

```
INSERT INTO Students (StudentID, Name, GPA)
VALUES (102, 'Asman', 3.50);
```

◇ UPDATE (Modify Data)

```
UPDATE Students
SET GPA = 3.60
WHERE StudentID = 102;
```

◇ DELETE (Remove Data)

```
DELETE FROM Students
WHERE StudentID = 102;
```

5️⃣ Primary Key, Foreign Key, Joins

🔑 Primary Key (PK)

- Uniquely identifies each row
- Cannot be NULL
- No duplicate values

Example: StudentID

🔗 Foreign Key (FK)

- Connects two tables
- Refers to Primary Key of another table

Example:

Students Table
Courses Table → StudentID (FK)

🔄 Joins (Combine Tables)

INNER JOIN Example:

```
SELECT Students.Name, Courses.CourseName
FROM Students
```

```
INNER JOIN Courses
ON Students.StudentID = Courses.StudentID;
```

👉 Combines data from two tables.

6️⃣ Tools Used for Database Testing

As you are learning QA, this is important 👇

◇ Database Tools

- MySQL Workbench
- SQL Server Management Studio
- phpMyAdmin

Used for:

- Running queries
- Checking data
- Verifying results

◇ QA Tools Used for Database Testing

- Selenium (Frontend + DB validation)
- JMeter (DB performance testing)
- Postman (API + DB validation)

🎓 Short Viva

- SQL database stores data in tables.
- MySQL is an open-source relational database.

- Table → Rows → Columns.
- Basic queries: SELECT, INSERT, UPDATE, DELETE.
- Primary Key = Unique identifier.
- Foreign Key = Connects tables.
- JOIN = Combine tables.
- Tools: MySQL Workbench, SSMS, phpMyAdmin.

EXTRA

1 ACID vs BASE

ACID (Used in Relational Databases)

ACID ensures **reliable and safe transactions**.

A – Atomicity

Transaction is **all or nothing**.
Example: Money transfer — if one step fails, everything is cancelled.

C – Consistency

Database always remains in a **valid state**.

I – Isolation

Multiple transactions don't affect each other.

D – Durability

Once data is saved, it will not be lost (even after crash).

👉 Used in:
MySQL,
PostgreSQL

🟢 BASE (Used in NoSQL)

BASE focuses on **availability and scalability**.

B – Basically Available

System is always available.

A – Soft State

Data may change over time (not immediately consistent).

S – Eventually Consistent

Data will become consistent **after some time**.

👉 Used in:
MongoDB,
Cassandra

💧 ACID vs BASE Difference

ACID	BASE
Strong consistency	Eventual consistency
More strict	More flexible
Best for banking	Best for social media / big data

2️⃣ When to Use Relational vs NoSQL

☑️ Use Relational Database When:

- You need strong data consistency

- Data is structured
- Complex joins are required
- Example: Banking, University, ERP

Example software:
Microsoft SQL Server

☑ Use NoSQL When:

- Data is unstructured or semi-structured
- System needs high scalability
- Large real-time applications
- Example: Chat apps, Social media

Example software:
Redis

3 Important Interview Questions (With Short Answers)

Q1: What is the difference between SQL and NoSQL?

👉 SQL is table-based with fixed schema.
👉 NoSQL is flexible and non-tabular.

Q2: What is a Primary Key?

👉 A unique identifier for each row in a table.

Q3: What is a Foreign Key?

👉 A key that connects one table to another.

Q4: What is ACID?

👉 A property ensuring reliable database transactions.

Q5: Why is NoSQL scalable?

👉 Because it supports horizontal scaling (adding more servers).

Q6: What is normalization?

👉 Process of organizing data to reduce redundancy.

🎓 Final Short Summary (Exam Style)

- **Relational Database** → Table-based, structured, uses SQL, follows ACID.
- **NoSQL Database** → Flexible schema, scalable, follows BASE.
- ACID = Strong consistency.
- BASE = High availability & scalability.